

Parsing

Chapter 3

Outline: How to build a parser?

- Specifying the syntax of a programming language with Context-Free Grammars (CFG)
- Build the parser based on the CFG:
 - Top-Down Parsing
 - **Predictive Parsing** (LL(k) Parsing)
 - Bottom-Up Parsing
- More about parsing:
 - Automatic parser generation
 - Error Recovery

3.2 Predictive Parsing

Types of Parsers for Grammars

- There are three general types of parsers for grammars
 - Universal
 - Top-Down
 - Bottom-Up
- **Universal parsing**
 - can parse any grammar
 - is too **inefficient** to use in production compilers
- **Top-Down parsing** builds parse trees from the top (root) to the bottom (leaves)
- **Bottom-Up parsing** builds parse trees from the leaves to the root.

Types of Parsers for Grammars

- Both Top-Down and Bottom-Up parsing methods scan the input from **left to right**, one symbol at a time.
- The most efficient top-down and bottom-up methods work **only for subclasses of grammars**.
- But several of these classes, particularly, LL and LR grammars, are **expressive enough** to describe most of the syntactic constructs in modern programming languages.
- Parsers implemented **by hand** often use **LL grammars**.
- Parsers for the larger class of **LR grammars** are usually constructed using **automated tools**.

Top-Down Parsing

- **Top-down parsing:**
 - the parse tree is constructed from the top, from left to right
 - can be viewed as finding a leftmost derivation for an input string

Top-Down Parsing

- Top-Down parsing For example,
- Grammar:

S $\rightarrow$ if E then S else S	L $\rightarrow$ end
S $\rightarrow$ begin S L	L $\rightarrow$; S L
S $\rightarrow$ print E	E $\rightarrow$ num = num

Grammar 3.11.

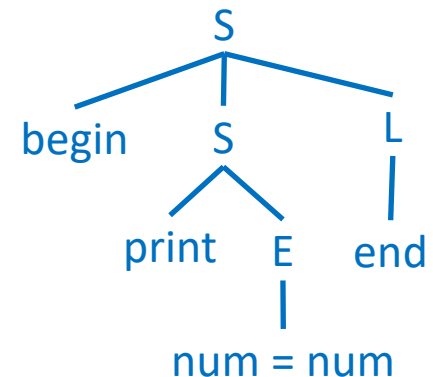
- String: ***begin print num = num end***
- From top to bottom, from left to right:

S $\rightarrow$ begin S L

begin S L $\rightarrow$ begin print E L

begin print E L $\rightarrow$ begin print num = num L

begin print num = num L $\rightarrow$ begin print num = num end



Top-Down Parsing

- **Top-down parsing:**

- the parse tree is constructed from the top, from left to right
- can be viewed as finding a leftmost derivation for an input string

- **Recursive descent Parsing**

- a general form of top-down parsing
- simple, can be coded by hand
- may require backtracing to find the correct production to be applied.

Recursive-Descent Parsing

- Write a recursive-descent parser for this grammar.
- Key ideas:
 - one recursive function for each non-terminal -> call this function to match this non-terminal
 - each production becomes one clause in the function

S -> if E then S else S

L -> end

S -> begin S L

L -> ; S L

S -> print E

E -> num = num

Grammar 3.11.

```
void L(void) {  
    switch(tok) {  
        case END: eat(END); break;  
        case SEMI: eat(SEMI); S(); L();  
    break;  
        default: error();  
    }  
}
```

Recursive-Descent Parsing

- Write the parser

S -> if E then S else S

S -> begin S L

S -> print E

L -> end

L -> ; S L

E -> num = num

- Step 1: Represent the token

```
enum token {IF, THEN, ELSE, BEGIN, END, PRINT, SEMI,  
NUM, EQ};
```

Recursive-Descent Parsing

- Write the parser

S -> if E then S else S

L -> end

S -> begin S L

L -> ; S L

S -> print E

E -> num = num

- Step 2: build infrastructure for reading tokens from lexer

```
// call lexer
```

```
extern enum token getToken(void);
```

```
// store the next token
```

```
enum token tok;
```

```
void advance() {tok=getToken();}
```

```
// consume the next token and get the new one
```

```
void eat(enum token t) {if (tok==t) advance(); else  
error();}
```

Recursive-Descent Parsing

- Write the parser

S -> if E then S else S

L -> end

S -> begin S L

L -> ; S L

S -> print E

E -> num = num

- Step 3: build a function for each non-terminal

```
void S(void) {  
    switch(tok) {  
        case IF: eat(IF); E(); eat(THEN); S(); eat(ELSE); S(); break;  
        case BEGIN: eat(BEGIN); S(); L(); break;  
        case PRINT: eat(PRINT); E(); break;  
        default: error(); }  
}  
void L(void) {  
    switch(tok) {  
        case END: eat(END); break;  
        case SEMI: eat(SEMI); S(); L(); break;  
        default: error(); }  
}  
void E(void) { eat(NUM); eat(EQ); eat(NUM); }
```

Recursive-Descent Parsing

```
enum token {IF, THEN, ELSE, BEGIN, END, PRINT, SEMI, NUM, EQ};
extern enum token getToken(void);
enum token tok;
void advance() {tok=getToken();}
void eat(enum token t) {if (tok==t) advance(); else error();}

void S(void) {
    switch(tok) {
        case IF: eat(IF); E(); eat(THEN); S(); eat(ELSE); S(); break;
        case BEGIN: eat(BEGIN); S(); L(); break;
        case PRINT: eat(PRINT); E(); break;
        default: error(); }}
void L(void) {
    switch(tok) {
        case END: eat(END); break;
        case SEMI: eat(SEMI); S(); L(); break;
        default: error(); }}
void E(void) { eat(NUM); eat(EQ); eat(NUM); }
```

Recursive-Descent Parsing

- This grammar is very special
 - For each non-terminal, the first symbol **Y1** in the right-hand sides of its productions are **different terminals**

S -> if E then S else S

S -> begin S L

S -> print E

L -> end

L -> ; S L

E -> num = num

- If there is another production:
 - **S -> S L** When **tok** is **begin**, which production
 - **S -> begin E, S L** to be applied to replace **S**?

Recursive-Descent Parsing

- How to write the parser for the following language?

$S \rightarrow E\$$

$E \rightarrow E + T$

$E \rightarrow E - T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow T / F$

$T \rightarrow F$

$F \rightarrow \text{id}$

$F \rightarrow \text{num}$

$F \rightarrow (E)$

- Step 3: build a function for each non-terminal

```
void S(void) { E(); eat(E0F); }
```

```
void E(void) {
```

```
    switch (tok) {
```

```
        case ? : E(); eat(PLUS); T(); break;
```

```
        case ? : E(); eat(MINUS); T(); break;
```

```
        case ? : T(); break;
```

```
        default: error();
```

```
    }
```

```
}
```

If $\text{tok} == \text{num}$,
which production
to choose?

Backtracking?

Recursive Descent Parsing

- If `tok == num`, which production to choose?
- Require **backtracking**
 - Too expensive
 - The number of paths attempted may explode exponentially
- Is there any recursive descent parsing method that does not require backtracking?

Top-Down Parsing

- **Top-down parsing:**

- the parse tree is constructed from the top, from left to right
- can be viewed as finding a leftmost derivation for an input string

- **Recursive descent Parsing**

- a general form of top-down parsing
- simple, can be coded by hand
- may require backtracing to find the correct production to be applied.

- **Predictive parsing:**

- a special case of recursive descent parsing
- no backtracking is required
- chooses the correct production by looking ahead at the input a fixed number of symbols (typically only 1)
- can parse $LL(k)$ grammars: Left-to-right parse; Leftmost-derivation; k symbol lookahead

Predictive Parsing

- Predictive parsing works only on grammars where we can determine which production to use by looking at the input k symbols.

$E \rightarrow E + T$

$E \rightarrow E - T$

$E \rightarrow T$

tok is a terminal symbol

- Q: Suppose $k=1$, if $tok == num$, which production to choose?
 - First, we need to know the possible first terminal symbols after deriving each of the three productions
 - Second, if the possible first terminal symbols of multiple productions can be num , we need to rewrite the grammar so that only one production can be use.

Predictive Parsing – First and Follow Sets

- Q: Suppose $k=1$, if $\text{tok} == \text{num}$, which production to choose?
 - First, we need to know the possible first terminal symbols after deriving each of the three productions -> **How?**

Predictive Parsing – First and Follow Sets

- γ, β, δ : a string of terminal and nonterminal symbols
- t : a terminal symbol
- X : a non-terminal symbol
- $\text{FIRST}(\gamma)$: if $\gamma \rightarrow^* t\beta$, then $t \in \text{First}(\gamma)$
 - γ can derive a t in the first position
 - **$\text{FIRST}(\gamma)$ is the set of terminals that can begin strings derived from γ**
- $\text{Follow}(X)$: if $S \rightarrow^* \beta X t \delta$, then $t \in \text{Follow}(X)$
 - If the derivation contains $XYZt$ where Y and Z both can derive ϵ , $t \in \text{Follow}(X)$
 - **$\text{FOLLOW}(X)$ is the set of terminals that can immediately follow X .**

Predictive Parsing – First and Follow Sets

- Q: Suppose $k=1$, if $\text{tok} == \text{num}$, which production to choose?
 - First, we need to know the possible first terminal symbols after deriving each of the three productions -> **How?**
- Consider non-terminal X , production $X \rightarrow \gamma$, possible first terminal symbol t can be:
 - any token in $\text{First}(\gamma)$
 - any token in $\text{Follow}(X)$ if $\gamma \xrightarrow{*} \varepsilon$
- Understanding $\text{First}(\gamma)$ and $\text{Follow}(X)$

Predictive Parsing – First and Follow Sets

First(X) (First Set of X) is computed iteratively:

- **Base case:**
 - if X is a terminal symbol, $\text{First}(X) = \{X\}$
 - if X is a non-terminal symbol, initialize $\text{First}(X) = \{\}$
- **Inductive case:** if X is a non-terminal symbol and $X \rightarrow Y_1 Y_2 \dots Y_k$, $\text{First}(X) = \text{First}(X) \cup \text{First}(Y_1 Y_2 \dots Y_k) = \text{First}(X) \cup F_1 \cup F_2 \cup \dots \cup F_k$, where:
 - $F_1 = \text{First}(Y_1)$
 - $F_2 = \text{First}(Y_2)$ if $Y_1 \xrightarrow{*} \varepsilon$, otherwise $\emptyset$
 - ...
 - $F_k = \text{First}(Y_k)$ if $Y_1 Y_2 \dots Y_{k-1} \xrightarrow{*} \varepsilon$, otherwise $\emptyset$

Predictive Parsing – First and Follow Sets

Follow(X) (Follow Set of X) is computed iteratively:

- **Base case:** initially, we assume nothing follows X, i.e., initialize $\text{Follow}(X) = \{\}$
- **Inductive case:**
 - For any strings α, β , if $Y \rightarrow \alpha X \beta$, then $\text{Follow}(X) = \text{Follow}(X) \cup \text{First}(\beta)$
 - For any strings α, β , if $Y \rightarrow \alpha X \beta$ and $\beta \xrightarrow{*} \varepsilon$, $\text{Follow}(X) = \text{Follow}(X) \cup \text{Follow}(Y)$

It is important to know whether a symbol can derive ε .

Predictive Parsing – First and Follow Sets

Computing Nullable Symbols

- $\text{Nullable}(X) = \text{True}$ if $X \xrightarrow{*} \epsilon$
- **How to compute?**
 - Iteration, similar to ϵ -closure
- Algorithm:

for each symbol X :

$\text{Nullable}(X) = \text{False}$

for each production $X \rightarrow Y_1 Y_2 \dots Y_k$:

 if $\text{Nullable}(Y_i) = \text{True}$ for $1 \leq i \leq k$:

$\text{Nullable}(X) = \text{True}$

Predictive Parsing – First and Follow Sets

Computing Nullable Symbols

- $\text{Nullable}(X) = \text{True}$ if $X \xrightarrow{*} \epsilon$
- **How to compute?**
 - Iteration, similar to ϵ -closure
- **Algorithm:**

for each symbol X :

$\text{Nullable}(x) = \text{False}$

repeat

 for each production $X \rightarrow Y_1 Y_2 \dots Y_k$:

 if $\text{Nullable}(Y_i) = \text{True}$ for $1 \leq i \leq k$:

$\text{Nullable}(X) = \text{True}$

until Nullable did not change in this iteration

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow X Y Z$
 $Z \rightarrow d$

$Y \rightarrow c$
 $Y \rightarrow$

$X \rightarrow a$
 $X \rightarrow Y$

	Nullable	First	Follow
Z			
Y			
X			

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow X Y Z$
 $Z \rightarrow d$

$Y \rightarrow c$
 $Y \rightarrow$

$X \rightarrow a$
 $X \rightarrow Y$

	Nullable	First	Follow
Z	False		
Y	False		
X	False		

Initialize nullable

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow XYZ$

$Z \rightarrow d$

$Y \rightarrow c$

$Y \rightarrow$

$X \rightarrow a$

$X \rightarrow Y$

	Nullable	First	Follow
Z	False		
Y	False		
X	False		

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow XYZ$

$Z \rightarrow d$

$Y \rightarrow c$

$Y \rightarrow$

$X \rightarrow a$

$X \rightarrow Y$

	Nullable	First	Follow
Z	False		
Y	False		
X	False		

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow X Y Z$
 $Z \rightarrow d$

$Y \rightarrow c$

$Y \rightarrow$

$X \rightarrow a$

$X \rightarrow Y$

	Nullable	First	Follow
Z	False		
Y	True		
X	False		

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow XYZ$
 $Z \rightarrow d$

$Y \rightarrow c$
 $Y \rightarrow$

$X \rightarrow a$
 $X \rightarrow Y$

	Nullable	First	Follow
Z	False		
Y	True		
X	True		

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow XYZ$

$Z \rightarrow d$

$Y \rightarrow c$

$Y \rightarrow$

$X \rightarrow a$

$X \rightarrow Y$

	Nullable	First	Follow
Z	False		
Y	True		
X	True		

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow XYZ$

$Z \rightarrow d$

$Y \rightarrow c$

$Y \rightarrow$

$X \rightarrow a$

$X \rightarrow Y$

	Nullable	First	Follow
Z	False		
Y	True		
X	True		

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow X Y Z$
 $Z \rightarrow d$

$Y \rightarrow c$

$Y \rightarrow$

$X \rightarrow a$

$X \rightarrow Y$

	Nullable	First	Follow
Z	False		
Y	True		
X	True		

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow X Y Z$
 $Z \rightarrow d$

$Y \rightarrow c$
 $Y \rightarrow$

$X \rightarrow a$
 $X \rightarrow Y$

	Nullable	First	Follow
Z	False		
Y	True		
X	True		

Stop iteration

Tips

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow X Y Z$
 $Z \rightarrow d$

$Y \rightarrow c$
 $Y \rightarrow$

$X \rightarrow a$
 $X \rightarrow Y$

	Nullable	First	Follow
Z	False	{ }	
Y	True	{ }	
X	True	{ }	

Initialize First sets

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow XYZ$

$Z \rightarrow d$

$Y \rightarrow c$

$Y \rightarrow$

$X \rightarrow a$

$X \rightarrow Y$

	Nullable	First	Follow
Z	False	{ }	
Y	True	{ }	
X	True	{ }	

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow XYZ$

$Z \rightarrow d$

$Y \rightarrow c$

$Y \rightarrow$

$X \rightarrow a$

$X \rightarrow Y$

	Nullable	First	Follow
Z	False	{d}	
Y	True	{c}	
X	True	{a}	

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow X Y Z$
 $Z \rightarrow d$

$Y \rightarrow c$

$Y \rightarrow$

$X \rightarrow a$

$X \rightarrow Y$

	Nullable	First	Follow
Z	False	{d}	
Y	True	{c}	
X	True	{a}	

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow X Y Z$
 $Z \rightarrow d$

$Y \rightarrow c$
 $Y \rightarrow$

$X \rightarrow a$
 $X \rightarrow Y$

	Nullable	First	Follow
Z	False	{d}	
Y	True	{c}	
X	True	{a, c}	

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow XYZ$

$Z \rightarrow d$

$Y \rightarrow c$

$Y \rightarrow$

$X \rightarrow a$

$X \rightarrow Y$

	Nullable	First	Follow
Z	False	{d, a, c}	
Y	True	{c}	
X	True	{a, c}	

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow XYZ$

$Z \rightarrow d$

$Y \rightarrow c$

$Y \rightarrow$

$X \rightarrow a$

$X \rightarrow Y$

	Nullable	First	Follow
Z	False	{d, a, c}	
Y	True	{c}	
X	True	{a, c}	

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow X Y Z$
 $Z \rightarrow d$

$Y \rightarrow c$
 $Y \rightarrow$

$X \rightarrow a$
 $X \rightarrow Y$

	Nullable	First	Follow
Z	False	{d, a, c}	
Y	True	{c}	
X	True	{a, c}	

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow X Y Z$
 $Z \rightarrow d$

$Y \rightarrow c$
 $Y \rightarrow$

$X \rightarrow a$
 $X \rightarrow Y$

	Nullable	First	Follow
Z	False	{d, a, c}	
Y	True	{c}	
X	True	{a, c}	

Iterate again.
No change.
Stop iteration.

Tips

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow X Y Z$
 $Z \rightarrow d$

$Y \rightarrow c$
 $Y \rightarrow$

$X \rightarrow a$
 $X \rightarrow Y$

	Nullable	First	Follow
Z	False	{d, a, c}	{ }
Y	True	{c}	{ }
X	True	{a, c}	{ }

Initialize follow sets

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow XYZ$
 $Z \rightarrow d$

$Y \rightarrow c$
 $Y \rightarrow$

$X \rightarrow a$
 $X \rightarrow Y$

	Nullable	First	Follow
Z	False	{d, a, c}	{ }
Y	True	{c}	{ }
X	True	{a, c}	{ }

Initialize follow sets

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow XYZ$
 $Z \rightarrow d$

$Y \rightarrow c$
 $Y \rightarrow$

$X \rightarrow a$
 $X \rightarrow Y$

	Nullable	First	Follow
Z	False	{d, a, c}	{ }
Y	True	{c}	{d, a, c}
X	True	{a, c}	{c, d, a}

First(Z) = ?

First(YZ) = ?

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow XYZ$

$Z \rightarrow d$

$Y \rightarrow c$

$Y \rightarrow$

$X \rightarrow a$

$X \rightarrow Y$

	Nullable	First	Follow
Z	False	{d, a, c}	{ }
Y	True	{c}	{d, a, c}
X	True	{a, c}	{c, d, a}

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow X Y Z$
 $Z \rightarrow d$

$Y \rightarrow c$
 $Y \rightarrow$

$X \rightarrow a$
 $X \rightarrow Y$

	Nullable	First	Follow
Z	False	{d, a, c}	{}
Y	True	{c}	{d, a, c}
X	True	{a, c}	{c, d, a}

Follow(Y) =
Follow(Y) $\cup$
Follow(X)

Predictive Parsing – First and Follow Sets

- To know the possible first terminal symbols of each production, we need to calculate the Nullable, First and Follow Sets

$Z \rightarrow X Y Z$
 $Z \rightarrow d$

$Y \rightarrow c$
 $Y \rightarrow$

$X \rightarrow a$
 $X \rightarrow Y$

	Nullable	First	Follow
Z	False	{d, a, c}	{}
Y	True	{c}	{d, a, c}
X	True	{a, c}	{c, d, a}

Iterate again.
No change.
Stop iteration.

Predictive Parsing – First and Follow Sets

- The algorithm to calculate First, Follow and Nullable Sets

Initialize FIRST and FOLLOW to all empty sets, and nullable to all false.

for each terminal symbol Z

$\text{FIRST}[Z] \leftarrow \{Z\}$

repeat

for each production $X \rightarrow Y_1 Y_2 \cdots Y_k$ **for** each production

for each i from 1 to k , each j from $i + 1$ to k ,

if all the Y_i are nullable

then $\text{nullable}[X] \leftarrow \text{true}$

if $Y_1 \cdots Y_{i-1}$ are all nullable

then $\text{FIRST}[X] \leftarrow \text{FIRST}[X] \cup \text{FIRST}[Y_i]$

if $Y_{i+1} \cdots Y_k$ are all nullable

then $\text{FOLLOW}[Y_i] \leftarrow \text{FOLLOW}[Y_i] \cup \text{FOLLOW}[X]$

if $Y_{i+1} \cdots Y_{j-1}$ are all nullable

then $\text{FOLLOW}[Y_i] \leftarrow \text{FOLLOW}[Y_i] \cup \text{FIRST}[Y_j]$

until FIRST, FOLLOW, and nullable did not change in this iteration.

Predictive Parsing – First and Follow Sets

- The algorithm to calculate First, Follow and Nullable Sets

Initialize FIRST and FOLLOW to all empty sets, and nullable to all false.

for each terminal symbol Z

$\text{FIRST}[Z] \leftarrow \{Z\}$

repeat

for each production $X \rightarrow Y_1 Y_2 \cdots Y_k$

for each i from 1 to k , each j from $i + 1$ to k ,

if all the Y_i are nullable

then $\text{nullable}[X] \leftarrow \text{true}$

compute nullable iteratively

if $Y_1 \cdots Y_{i-1}$ are all nullable

then $\text{FIRST}[X] \leftarrow \text{FIRST}[X] \cup \text{FIRST}[Y_i]$

if $Y_{i+1} \cdots Y_k$ are all nullable

then $\text{FOLLOW}[Y_i] \leftarrow \text{FOLLOW}[Y_i] \cup \text{FOLLOW}[X]$

if $Y_{i+1} \cdots Y_{j-1}$ are all nullable

then $\text{FOLLOW}[Y_i] \leftarrow \text{FOLLOW}[Y_i] \cup \text{FIRST}[Y_j]$

until FIRST, FOLLOW, and nullable did not change in this iteration.

Predictive Parsing – First and Follow Sets

- The algorithm to calculate First, Follow and Nullable Sets

Initialize FIRST and FOLLOW to all empty sets, and nullable to all false.

for each terminal symbol Z

$\text{FIRST}[Z] \leftarrow \{Z\}$

repeat

for each production $X \rightarrow Y_1 Y_2 \cdots Y_k$

for each i from 1 to k , each j from $i + 1$ to k ,

if all the Y_i are nullable

then $\text{nullable}[X] \leftarrow \text{true}$ Analyze each prefix $Y_1 \dots Y_{i-1}$

if $Y_1 \cdots Y_{i-1}$ are all nullable

then $\text{FIRST}[X] \leftarrow \text{FIRST}[X] \cup \text{FIRST}[Y_i]$

if $Y_{i+1} \cdots Y_k$ are all nullable

then $\text{FOLLOW}[Y_i] \leftarrow \text{FOLLOW}[Y_i] \cup \text{FOLLOW}[X]$

if $Y_{i+1} \cdots Y_{j-1}$ are all nullable

then $\text{FOLLOW}[Y_i] \leftarrow \text{FOLLOW}[Y_i] \cup \text{FIRST}[Y_j]$

until FIRST, FOLLOW, and nullable did not change in this iteration.

Predictive Parsing – First and Follow Sets

- The algorithm to calculate First, Follow and Nullable Sets

Initialize FIRST and FOLLOW to all empty sets, and nullable to all false.

for each terminal symbol Z

$\text{FIRST}[Z] \leftarrow \{Z\}$

repeat

for each production $X \rightarrow Y_1 Y_2 \cdots Y_k$

for each i from 1 to k , each j from $i + 1$ to k ,

if all the Y_i are nullable

then $\text{nullable}[X] \leftarrow \text{true}$

if $Y_1 \cdots Y_{i-1}$ are all nullable Analyze each suffix $Y_{i+1} \cdots Y_k$

then $\text{FIRST}[X] \leftarrow \text{FIRST}[X] \cup \text{FIRST}[Y_i]$

if $Y_{i+1} \cdots Y_k$ are all nullable

then $\text{FOLLOW}[Y_i] \leftarrow \text{FOLLOW}[Y_i] \cup \text{FOLLOW}[X]$

if $Y_{i+1} \cdots Y_{j-1}$ are all nullable

then $\text{FOLLOW}[Y_i] \leftarrow \text{FOLLOW}[Y_i] \cup \text{FIRST}[Y_j]$

until FIRST, FOLLOW, and nullable did not change in this iteration.

Predictive Parsing – First and Follow Sets

- The algorithm to calculate First, Follow and Nullable Sets

Initialize FIRST and FOLLOW to all empty sets, and nullable to all false.

for each terminal symbol Z

$\text{FIRST}[Z] \leftarrow \{Z\}$

$X \rightarrow Y_1 Y_2 \dots Y_i \dots Y_j \dots Y_k$

repeat

this substring

for each production $X \rightarrow Y_1 Y_2 \dots Y_k$

for each i from 1 to k , each j from $i + 1$ to k ,

if all the Y_i are nullable

Analyze each **substring**,
 $Y_{i+1} \dots Y_{j-1}$ can be **empty**

then $\text{nullable}[X] \leftarrow \text{true}$

if $Y_1 \dots Y_{i-1}$ are all nullable

then $\text{FIRST}[X] \leftarrow \text{FIRST}[X] \cup \text{FIRST}[Y_i]$

if $Y_{i+1} \dots Y_k$ are all nullable

then $\text{FOLLOW}[Y_i] \leftarrow \text{FOLLOW}[Y_i] \cup \text{FOLLOW}[X]$

if $Y_{i+1} \dots Y_{j-1}$ are all nullable

then $\text{FOLLOW}[Y_i] \leftarrow \text{FOLLOW}[Y_i] \cup \text{FIRST}[Y_j]$

until FIRST, FOLLOW, and nullable did not change in this iteration.

Predictive Parsing – First and Follow Sets

- The algorithm to calculate First, Follow and Nullable Sets

Initialize FIRST and FOLLOW to all empty sets, and nullable to all false.

for each terminal symbol Z

$\text{FIRST}[Z] \leftarrow \{Z\}$

repeat

for each production $X \rightarrow Y_1 Y_2 \cdots Y_k$

for each i from 1 to k , each j from $i + 1$ to k ,

if all the Y_i are nullable

Iteration

then $\text{nullable}[X] \leftarrow \text{true}$

if $Y_1 \cdots Y_{i-1}$ are all nullable

then $\text{FIRST}[X] \leftarrow \text{FIRST}[X] \cup \text{FIRST}[Y_i]$

if $Y_{i+1} \cdots Y_k$ are all nullable

then $\text{FOLLOW}[Y_i] \leftarrow \text{FOLLOW}[Y_i] \cup \text{FOLLOW}[X]$

if $Y_{i+1} \cdots Y_{j-1}$ are all nullable

then $\text{FOLLOW}[Y_i] \leftarrow \text{FOLLOW}[Y_i] \cup \text{FIRST}[Y_j]$

until FIRST, FOLLOW, and nullable did not change in this iteration.

Predictive Parsing – First and Follow Sets

- To make this algorithm efficient, it helps to examine the productions **in the right order**.
- The three relations **need not** be computed simultaneously.
 - **Nullable** first, then **FIRST**, then **FOLLOW**
- We convert **a group of equations on sets** to **the algorithm for calculating those sets**.
 - another example: computing **ϵ -closure**
 - **iteration to a fixed point**

Predictive Parsing – First and Follow Sets

- Wait, why do we need First and Follow set?
- Q: Suppose $k = 1$, if $\text{tok} == \text{num}$, which production to choose?
 - First, we need to know the possible first terminal symbols when we choose each of the three productions. -> **How?**

Predictive Parsing - Predictive Parsing Tables

Grammar:

$Z \rightarrow XYZ$ $Y \rightarrow c$ $X \rightarrow a$
 $Z \rightarrow d$ $Y \rightarrow$ $X \rightarrow Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

Build parsing table where row X , col T tells parser which clause to execute in function X with next-token T :

- if $T \in \text{First}(\gamma)$ then
enter $(X \rightarrow \gamma)$ in row X , col T
- if γ is Nullable and $T \in \text{Follow}(X)$
enter $(X \rightarrow \gamma)$ in row X , col T

	a	c	d
Z			
Y			
X			

Predictive Parsing - Predictive Parsing Tables

Grammar:

$Z \rightarrow XYZ$ $Y \rightarrow c$ $X \rightarrow a$
 $Z \rightarrow d$ $Y \rightarrow$ $X \rightarrow Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

Build parsing table where row X , col T tells parser which clause to execute in function X with next-token T :

- if $T \in \text{First}(\gamma)$ then enter $(X \rightarrow \gamma)$ in row X , col T
- if γ is Nullable and $T \in \text{Follow}(X)$ enter $(X \rightarrow \gamma)$ in row X , col T

	a	c	d
Z	$Z \rightarrow XYZ$	$Z \rightarrow XYZ$	$Z \rightarrow XYZ$
Y			
X			

First(XYZ)

Predictive Parsing - Predictive Parsing Tables

Grammar:

$Z \rightarrow XYZ$

$Y \rightarrow c$

$X \rightarrow a$

$Z \rightarrow d$

$Y \rightarrow$

$X \rightarrow Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

Build parsing table where row X , col T tells parser which clause to execute in function X with next-token T :

- if $T \in \text{First}(\gamma)$ then enter $(X \rightarrow \gamma)$ in row X , col T
- if γ is Nullable and $T \in \text{Follow}(X)$ enter $(X \rightarrow \gamma)$ in row X , col T

	a	c	d
Z	$Z \rightarrow XYZ$	$Z \rightarrow XYZ$	$Z \rightarrow XYZ$ $Z \rightarrow d$
Y		$Y \rightarrow c$	
X	$X \rightarrow a$		

Predictive Parsing - Predictive Parsing Tables

Grammar:

$Z \rightarrow XYZ$ $Y \rightarrow c$ $X \rightarrow a$
 $Z \rightarrow d$ $Y \rightarrow$ $X \rightarrow Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

Build parsing table where row X , col T tells parser which clause to execute in function X with next-token T :

- if $T \in \text{First}(\gamma)$ then enter $(X \rightarrow \gamma)$ in row X , col T
- if γ is Nullable and $T \in \text{Follow}(X)$ enter $(X \rightarrow \gamma)$ in row X , col T

	a	c	d
Z	$Z \rightarrow XYZ$	$Z \rightarrow XYZ$	$Z \rightarrow XYZ$ $Z \rightarrow d$
Y	$Y \rightarrow$	$Y \rightarrow c$ $Y \rightarrow$	$Y \rightarrow$
X	$X \rightarrow a$		

Follow(Y)

Predictive Parsing - Predictive Parsing Tables

Grammar:

$Z \rightarrow XYZ$ $Y \rightarrow c$ $X \rightarrow a$
 $Z \rightarrow d$ $Y \rightarrow$ $X \rightarrow Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

Build parsing table where row X, col T tells parser which clause to execute in function X with next-token T:

- if $T \in \text{First}(\gamma)$ then enter $(X \rightarrow \gamma)$ in row X, col T
- if γ is Nullable and $T \in \text{Follow}(X)$ enter $(X \rightarrow \gamma)$ in row X, col T

	a	c	d
Z	Z->XYZ	Z->XYZ	Z->XYZ Z->d
Y	Y->	Y->c Y->	Y->
X	X->a X->Y	X->Y	X->Y

First(Y) $\cup$ Follow (X)

Predictive Parsing - Predictive Parsing Tables

Grammar:

$Z \rightarrow XYZ$ $Y \rightarrow c$ $X \rightarrow a$
 $Z \rightarrow d$ $Y \rightarrow$ $X \rightarrow Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

Build a predictive parsing table where
row X , col T tells parser which clause to
execute in function X with next-token T :

- if $T \in \text{First}(\gamma)$ then
enter $(X \rightarrow \gamma)$ in row X , col T
- if γ is Nullable and $T \in \text{Follow}(X)$
enter $(X \rightarrow \gamma)$ in row X , col T

	a	c	d	
Z	Z->XYZ	Z->XYZ	Z->XYZ Z->d	
Y	Y->	Y->c Y->	Y->	
X	X->a X->Y	X->Y	X->Y	

Predictive Parsing - Predictive Parsing Tables

Grammar:

$Z \rightarrow XYZ$ $Y \rightarrow c$ $X \rightarrow a$
 $Z \rightarrow d$ $Y \rightarrow$ $X \rightarrow Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

What does an empty entry mean?

syntax errors

	a	c	d
Z	Z->XYZ	Z->XYZ	Z->XYZ Z->d
Y	Y->	Y->c Y->	Y->
X	X->a X->Y	X->Y	X->Y

Predictive Parsing - Predictive Parsing Tables

Grammar:

$Z \rightarrow XYZ$ $Y \rightarrow c$ $X \rightarrow a$
 $Z \rightarrow d$ $Y \rightarrow$ $X \rightarrow Y$

	nullable	first	follow
Z	no	d,a,c	
Y	yes	c	a,c,d
X	yes	a,c	a,c,d

Is it ok if there are multiple grammar rules in the same entry?

Predictive parsing will not work based on this parsing table!

	a	c	d
Z	Z->XYZ	Z->XYZ	Z->XYZ Z->d
Y	Y->	Y->c Y->	Y->
X	X->a X->Y	X->Y	X->Y

Predictive Parsing - Predictive Parsing Tables

- **LL(1) grammar**: if a predictive parsing table constructed this way contains no duplicate entries, the grammar is called LL(1).
 - Left-to-right parse, left-most derivation, 1 symbol lookahead
- If not, the grammar is not LL(1)
- In LL(k) parsing tables, columns include every k-length sequence of terminals:

	aa	ab	ba	bb	ac	ca	...
X							

- No ambiguous grammar is LL(k) for any k.

Non-Recursive Predictive Parser

- $S \rightarrow (S) S \mid \varepsilon$

M[N, T]	(	)	\$
S	$S \rightarrow (S)S$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

- Based on this parsing table,
 - we can build a **recursive** predictive parser
 - We can also build a **non-recursive** predictive parser by maintaining a stack explicitly.
- See Dragon Book for the algorithm.
- For example: to parse **()**

Non-Recursive Predictive Parser

- $S \rightarrow (S) S \mid \varepsilon$

M[N, T]	(	)	\$
S	$S \rightarrow (S)S$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

Steps	Parsing Stack	Input	Action
1	\$S	()\$	

The symbol on top of the stack:

- Terminal:** Match
- Non-terminal:** Derive

Non-Recursive Predictive Parser

- $S \rightarrow (S) S \mid \varepsilon$

M[N, T]	(	)	\$
S	$S \rightarrow (S)S$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

Steps	Parsing Stack	Input	Action
1	\$S	()\$	$S \rightarrow (S)S$

The symbol on top of the stack:

- Terminal:** Match
- Non-terminal:** Derive

Non-Recursive Predictive Parser

- $S \rightarrow (S) S \mid \varepsilon$

M[N, T]	(	)	\$
S	$S \rightarrow (S)S$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

Steps	Parsing Stack	Input	Action
1	\$S	()\$	$S \rightarrow (S)S$
2	\$S)S(	()\$	

The symbol on top of the stack:

- **Terminal**: Match
- **Non-terminal**: Derive

Non-Recursive Predictive Parser

- $S \rightarrow (S) S \mid \varepsilon$

M[N, T]	(	)	\$
S	$S \rightarrow (S)S$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

Steps	Parsing Stack	Input	Action
1	\$S	()\$	$S \rightarrow (S)S$
2	\$S)S(	()\$	match

The symbol on top of the stack:

- **Terminal**: Match
- **Non-terminal**: Derive

Non-Recursive Predictive Parser

- $S \rightarrow (S) S \mid \varepsilon$

M[N, T]	(	)	\$
S	$S \rightarrow (S)S$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

Steps	Parsing Stack	Input	Action
1	\$S	()\$	$S \rightarrow (S)S$
2	\$S)S(	()\$	match
3	\$S)S	)\$	

The symbol on top of the stack:

- Terminal:** Match
- Non-terminal:** Derive

Non-Recursive Predictive Parser

- $S \rightarrow (S) S \mid \varepsilon$

M[N, T]	(	)	\$
S	$S \rightarrow (S)S$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

Steps	Parsing Stack	Input	Action
1	\$S	()\$	$S \rightarrow (S)S$
2	\$S)S(	()\$	match
3	\$S)S	)\$	$S \rightarrow \varepsilon$

The symbol on top of the stack:

- **Terminal**: Match
- **Non-terminal**: Derive

Non-Recursive Predictive Parser

- $S \rightarrow (S) S \mid \varepsilon$

M[N, T]	(	)	\$
S	$S \rightarrow (S)S$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

Steps	Parsing Stack	Input	Action
1	\$S	()\$	$S \rightarrow (S)S$
2	\$S)S(	()\$	match
3	\$S)S	)\$	$S \rightarrow \varepsilon$
4	\$S)	)\$	

The symbol on top of the stack:

- **Terminal**: Match
- **Non-terminal**: Derive

Non-Recursive Predictive Parser

- $S \rightarrow (S) S \mid \varepsilon$

M[N, T]	(	)	\$
S	$S \rightarrow (S)S$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

Steps	Parsing Stack	Input	Action
1	\$S	()\$	$S \rightarrow (S)S$
2	\$S)S(	()\$	match
3	\$S)S	)\$	$S \rightarrow \varepsilon$
4	\$S)	)\$	match

The symbol on top of the stack:

- **Terminal**: Match
- **Non-terminal**: Derive

Non-Recursive Predictive Parser

- $S \rightarrow (S) S \mid \varepsilon$

M[N, T]	(	)	\$
S	$S \rightarrow (S)S$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

Steps	Parsing Stack	Input	Action
1	\$S	()\$	$S \rightarrow (S)S$
2	\$S)S(	()\$	match
3	\$S)S	)\$	$S \rightarrow \varepsilon$
4	\$S)	)\$	match
5	\$S	\$	$S \rightarrow \varepsilon$

The symbol on top of the stack:

- **Terminal**: Match
- **Non-terminal**: Derive

Non-Recursive Predictive Parser

- $S \rightarrow (S) S \mid \varepsilon$

M[N, T]	(	)	\$
S	$S \rightarrow (S)S$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

Steps	Parsing Stack	Input	Action
1	\$S	()\$	$S \rightarrow (S)S$
2	\$S)S(	()\$	match
3	\$S)S	)\$	$S \rightarrow \varepsilon$
4	\$S)	)\$	match
5	\$S	\$	$S \rightarrow \varepsilon$
6	\$	\$	accept

The symbol on top of the stack:

- **Terminal**: Match
- **Non-terminal**: Derive

Predictive Parsing – Eliminate Left-Recursion

- Q: Suppose $k=1$, if $\text{tok} == \text{num}$, which production to choose?
 - ~~First, we need to know the possible first terminal symbols when we choose each of the three productions~~
 - Second, if the possible first terminal symbols of multiple productions can be `num`, we need to rewrite the grammar so that only one production can be use.

Predictive Parsing – Eliminate Left-Recursion

- **Left Recursion**

- $E \rightarrow E + T$
 - $E \rightarrow T$
- $\{T, T + T, T + T + T, T + T + T + T, \dots\}$

- Cause duplicate entries in the LL(1) parsing table

- Why?

- $E \rightarrow E+T$: $\text{First}(E+T) \subseteq \text{First}(E)$
 - $\text{First}(E) \subseteq \text{First}(E+T)$
 - $\Rightarrow \text{First}(E) = \text{First}(E+T)$
 - $E \rightarrow T$: $\text{First}(T) \subseteq \text{First}(E)$
 - $\Rightarrow$ if $a \in \text{First}(T)$, then $a \in \text{First}(E)$ and $a \in \text{First}(E+T)$
 - For any such a , we can choose any of the two productions
- **Top-down parsing methods cannot handle left-recursive grammars.**

Predictive Parsing – Eliminate Left-Recursion

- How to eliminate left-recursion?
- Rewrite the grammar so it parses the same language but with different productions

$$\begin{aligned} E &\rightarrow E + T \\ E &\rightarrow T \end{aligned}$$

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \\ E' &\rightarrow \end{aligned}$$

- In general

$$\begin{aligned} A &\rightarrow Aa \\ A &\rightarrow \beta \end{aligned}$$

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow aA' \\ A' &\rightarrow \end{aligned}$$

$\{\beta, \beta a, \beta aa, \dots\}$

Use right recursion to generate the repetitions

Predictive Parsing – Eliminate Left-Recursion

$$S \rightarrow E \$$$

$$E \rightarrow E + T$$

$$E \rightarrow E - T$$

$$E \rightarrow T$$

$$T \rightarrow T * F$$

$$T \rightarrow T / F$$

$$T \rightarrow F$$

$$F \rightarrow \text{id}$$

$$F \rightarrow \text{num}$$

$$F \rightarrow (E)$$

GRAMMAR 3.10.



$$S \rightarrow E \$$$

$$E \rightarrow T E'$$

$$E' \rightarrow + T E'$$

$$E' \rightarrow - T E'$$

$$E' \rightarrow$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T'$$

$$T' \rightarrow / F T'$$

$$T' \rightarrow$$

$$F \rightarrow \text{id}$$

$$F \rightarrow \text{num}$$

$$F \rightarrow (E)$$

GRAMMAR 3.15.

Predictive Parsing – Eliminate Left-Recursion

$S \rightarrow E \$$

$E \rightarrow T E'$

$E' \rightarrow + T E'$

$E' \rightarrow - T E'$

$E' \rightarrow \varepsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T'$

$T' \rightarrow / F T'$

$T' \rightarrow \varepsilon$

$F \rightarrow \text{id}$

$F \rightarrow \text{num}$

$F \rightarrow (E)$

	nullable	FIRST	FOLLOW
S	no	(id num	
E	no	(id num	) \$
E'	yes	+ -	) \$
T	no	(id num	) + - \$
T'	yes	* /	) + - \$
F	no	(id num	) * / + - \$

	+	*	id	(	)	\$
S			$S \rightarrow E \$$	$S \rightarrow E \$$		
E			$E \rightarrow T E'$	$E \rightarrow T E'$		
E'	$E' \rightarrow + T E'$				$E' \rightarrow$	$E' \rightarrow$
T			$T \rightarrow F T'$	$T \rightarrow F T'$		
T'	$T' \rightarrow$	$T' \rightarrow * F T'$			$T' \rightarrow$	$T' \rightarrow$
F			$F \rightarrow \text{id}$	$F \rightarrow (E)$		

- is similar to +, / is similar to *, num is similar to id

Predictive Parsing – Eliminate Left-Recursion

- How to eliminate left-recursion?
- Rewrite the grammar so it parses the same language but with different productions

$$\begin{aligned} E &\rightarrow E + T \\ E &\rightarrow T \end{aligned}$$

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \\ E' &\rightarrow \end{aligned}$$

- In general

$$\begin{aligned} A &\rightarrow Aa \\ A &\rightarrow \beta \end{aligned}$$

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow aA' \\ A' &\rightarrow \end{aligned}$$

$\{\beta, \beta a, \beta aa, \dots\}$

Immediate left recursion

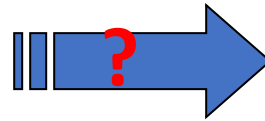
Use right recursion to
generate the repetitions

More generally, $A \xRightarrow{+} Aa$
How to Eliminate?

Predictive Parsing – Left Factoring

- If two productions for *the same non-terminal start with the same symbols*, the LL(1) parsing table will contain duplicate entries.
- Solution: **Left Factoring**, delay the decision.

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$
 $S \rightarrow \text{if } E \text{ then } S$



$S \rightarrow \text{if } E \text{ then } S X$
 $X \rightarrow \text{else } S$
 $X \rightarrow$

- In general

$A \rightarrow a\beta$
 $A \rightarrow a\gamma$
 $A \rightarrow a$



$A \rightarrow aA'$
 $A' \rightarrow \beta | \gamma | \epsilon$

Predictive Parsing – Error Recovery

- *A blank entry indicates a syntax error*
- How should errors be handled?
 - Raise an exception and quit parsing
 - Print an error message and recover from the error
- Raise an exception and quit:

```
void T( ) {  
    switch (tok) {  
        case ID:  
        case NUM:  
        case LPAREN: F(); Tprime(); break;  
        default: error!  
    }  
}
```

Predictive Parsing – Error Recovery

- Print an error message and recover from the error
- Errors can be recovered by deleting, replacing, or inserting tokens.
- **Through inserting:** pretend we have the token and return normally

```
void T( ) {  
    switch (tok) {  
        case ID: may not terminate  
        case NUM:  
        case LPAREN: F( ); Tprime( ); break;  
        default: print("expected id, num, or left-  
paren");  
    }  
}
```

- **Deleting tokens** is safer, because the loop must eventually terminate when EOF is reached

Predictive Parsing – Error Recovery

- Simple recover by deletion works by skipping tokens until a token in the FOLLOW set is reached.

```
int Tprime_follow [ ] = {PLUS, RPAREN, EOF};
void Tprime( ) {
    switch (tok) {
        case PLUS: break;
        case TIMES: eat(TIMES); F(); Tprime(); break;
        case RPAREN: break;
        case EOF: break;
        default: print("expected +, *, right-paren, or
end-of-file");
            skipto(Tprime_follow);
    }
}
```

Summary

- CFGs are good at specifying programming language structure
- We define parsers from simple classes of CFGs
 - LL(k), LR(k)
- We can build a predictive parser for LL(k) grammars by:
 - computing Nullable, First and Follow sets
 - constructing a parsing table from these sets
 - checking for duplicate entries, which indicates failure
 - creating an C program from the parsing table
- If parser construction fails, we can:
 - rewrite the grammar (left factoring, eliminating left recursion) and try again
 - try to build a parser using some other methods

Reminder

- 建议在下周上课前完成lab 1词法分析部分，并熟悉lab 1语法分析部分。
- 下周上完Bottom-Up Parsing后离lab 1截止日期将**不足1周**。